

EJERCICIO 4-2

ENTORNOS DE DESARROLLO

Juan Alonso Pérez Pérez

Ejercicio 1.	3
Ejercicio 2.	4
Ejercicio 3.	4
Ejercicio 4.	5
Ejercicio 5.	7
Ejercicio 6.	8
Ejercicio 7.	9
Ejercicio 8.	10

Ejercicio 1.

Cambio el nombre a la variable v por casa1:

```

10      {
11          Vivienda casa1 = new Vivienda( precio: 120000);
12
13          double precio;
14          int numHabitaciones;
15          double superficie;
16          boolean parking;
17          String estado;
18          String propietario;
19
20          System.out.println("Precio: " + casa1.precio);
21          System.out.println("Número de habitaciones: " + casa1.numHabitaciones);
22          System.out.println("Superficie en m2: " + casa1.superficie);
23          System.out.println("¿Dispone de parking?: " + casa1.parking);
24          System.out.println("Estado: " + casa1.estado);
25          System.out.println("Propietario: " + casa1.propietario);
26
27      }
28

```

Para cambiarle el nombre le doy click derecho, refactor, rename.

Ejercicio 2.

Hago este código pasándole un objeto al método imprimirVivienda, y luego después de poner los métodos de impresión los cambio el nombre:

```

public static void main(String[] args)
{
    Vivienda casa1 = new Vivienda( precio: 120000);

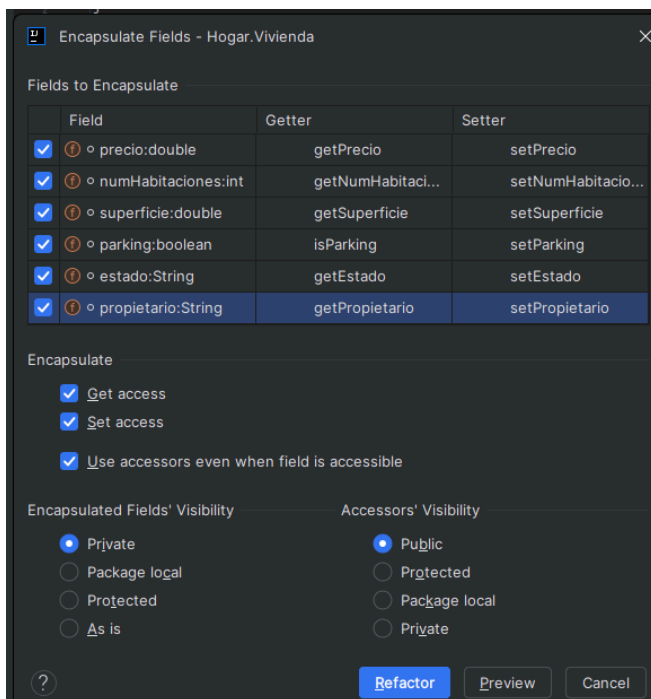
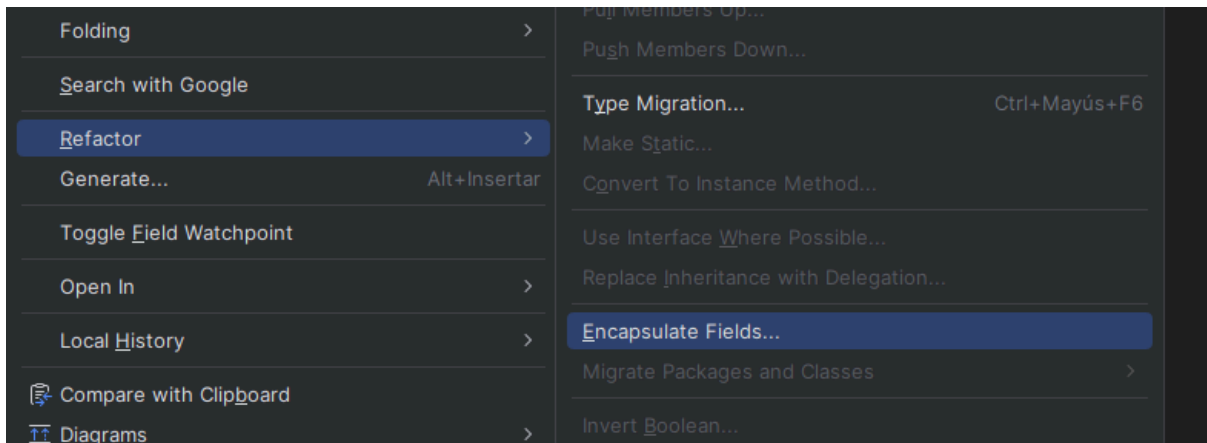
    double precio;
    int numHabitaciones;
    double superficie;
    boolean parking;
    String estado;
    String propietario;
    imprimirVivienda(casa1);
}

@ public static void imprimirVivienda(Vivienda v){ 1 usage
    System.out.println("Precio: " + v.precio);
    System.out.println("Número de habitaciones: " + v.numHabitaciones);
    System.out.println("Superficie en m2: " + v.superficie);
    System.out.println("¿Dispone de parking?: " + v.parking);
    System.out.println("Estado: " + v.estado);
    System.out.println("Propietario: " + v.propietario);
}
}

```

Ejercicio 3.

Encapsulo los atributos, los vuelvo privados y género los getters y los setters:



```
package Hogar;

public class Vivienda {
    private double precio;
    private int numHabitaciones;
    private double superficie;
    private boolean parking;
    private String estado;
    private String propietario;

    public Vivienda(double precio) { this.setPrecio(precio); }

    public double getPrecio() {
        return precio;
    }

    public void setPrecio(double precio) {
        this.precio = precio;
    }

    public int getNumHabitaciones() {
        return numHabitaciones;
    }

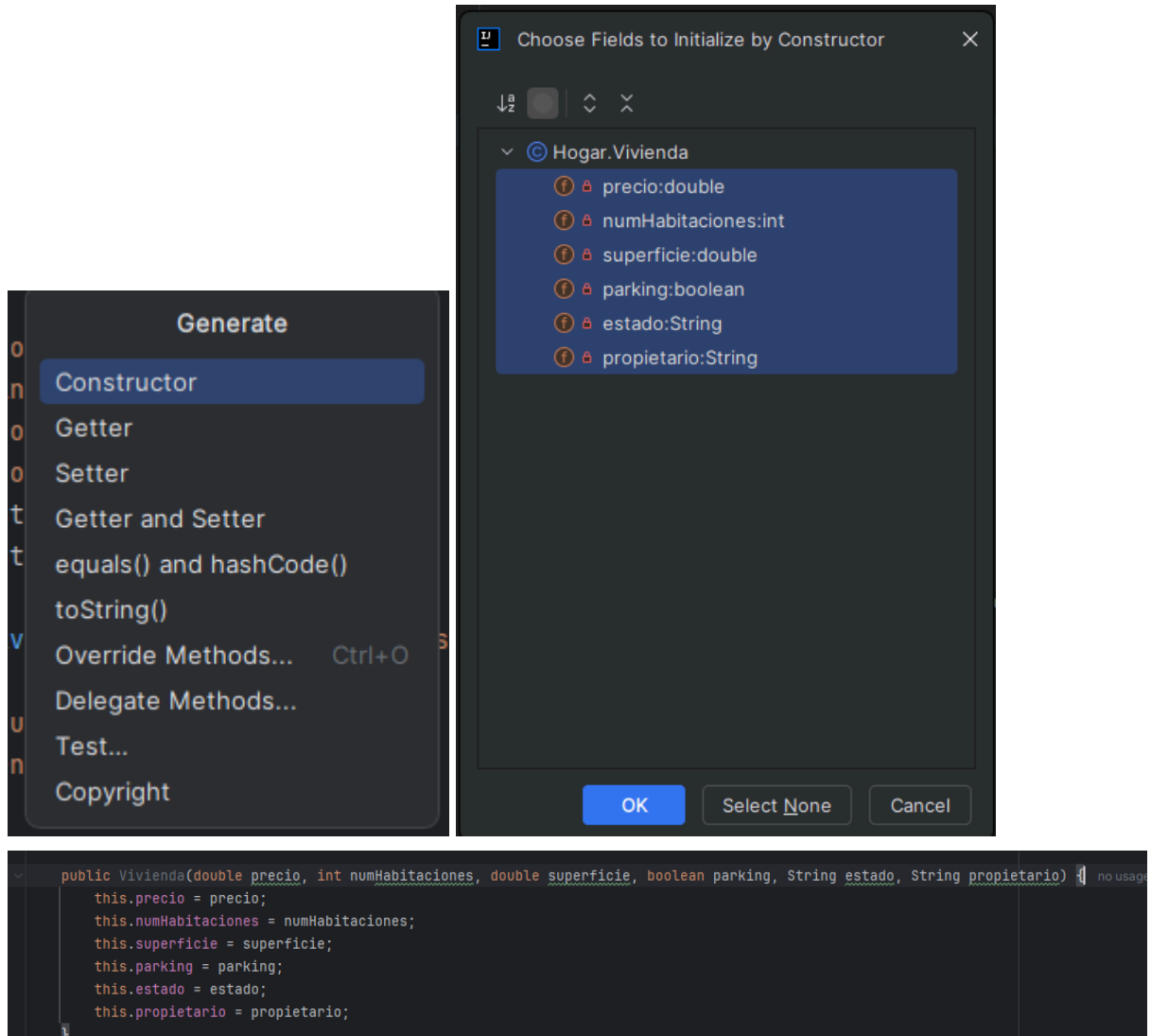
    public void setNumHabitaciones(int numHabitaciones) {
        this.numHabitaciones = numHabitaciones;
    }

    public double getSuperficie() {
        return superficie;
    }

    public void setSuperficie(double superficie) {
        this.superficie = superficie;
    }
}
```

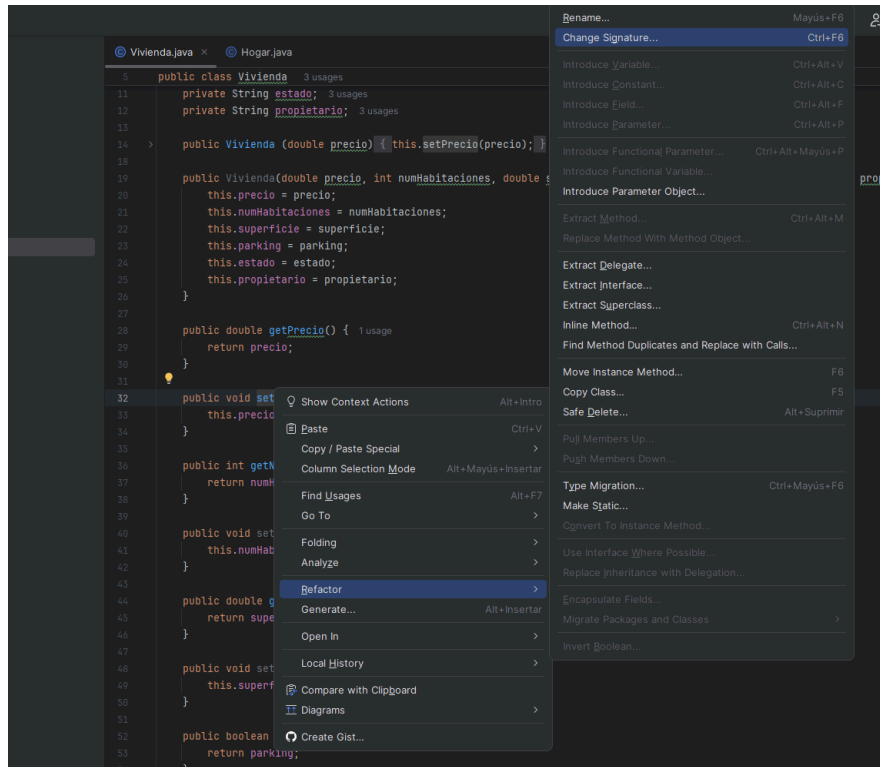
Ejercicio 4.

Creó el constructor con todos los parámetros:



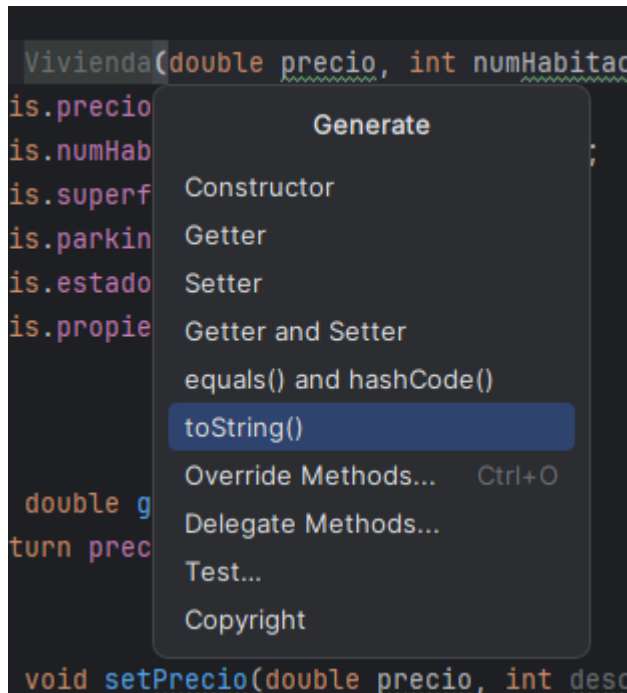
Ejercicio 5.

Cambio el método setPrecio:



Ejercicio 6.

Yo para optimizar el código crearé un método toString para luego no tener que crear un método para imprimir la vivienda:



```

Vivienda(double precio, int numHabitaciones, double superficie, boolean parking, String estado, String propietario, double descuento) {
    this.precio = precio;
    this.numHabitaciones = numHabitaciones;
    this.superficie = superficie;
    this.parking = parking;
    this.estado = estado;
    this.propietario = propietario;
    this.descuento = descuento;
}

double getPrecio() {
    return precio;
}

void setPrecio(double precio, int descuento) {
    this.precio = precio;
    this.descuento = descuento;
}
    
```

Generate

- Constructor
- Getter
- Setter
- Getter and Setter
- equals() and hashCode()
- toString()**
- Override Methods... Ctrl+O
- Delegate Methods...
- Test...
- Copyright

```

@Override
public String toString() {
    return "Vivienda{" +
        "precio=" + precio +
        ", numHabitaciones=" + numHabitaciones +
        ", superficie=" + superficie +
        ", parking=" + parking +
        ", estado='" + estado + '\'' +
        ", propietario='" + propietario + '\'' +
        ", descuento=" + descuento +
        '}';
}
    
```

```

public class Hogar {

    public static void main(String[] args) {
        Vivienda casa1 = new Vivienda(120000);

        double precio;
        int numHabitaciones;
        double superficie;
        boolean parking;
        String estado;
        String propietario;
        System.out.println(casa1);
    }
}
    
```

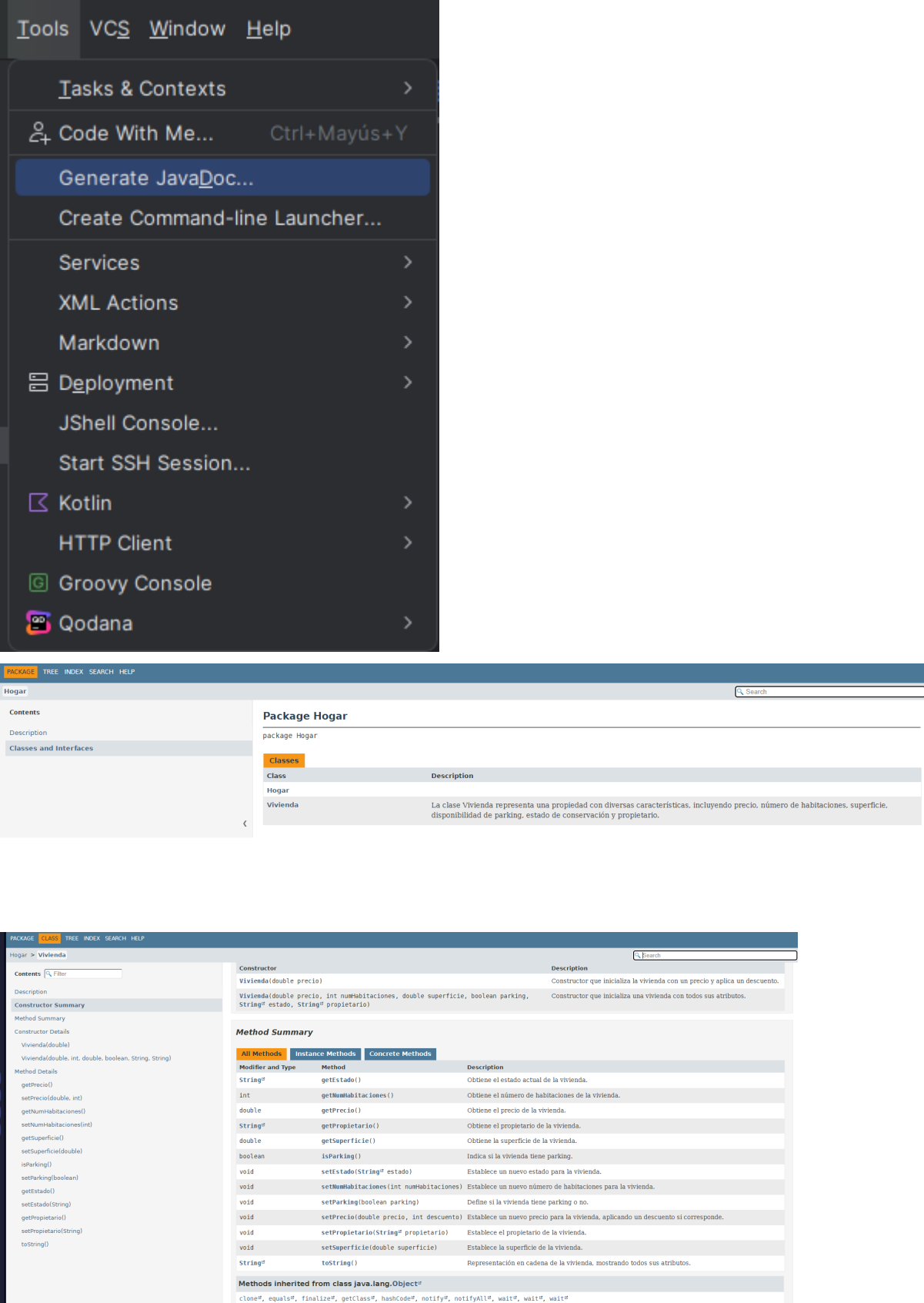
Ejercicio 7.

Creo los comentarios Javadoc:

```
1 package Hogar;
2
3 /**
4  * La clase Vivienda representa una propiedad con diversas características,
5  * incluyendo precio, número de habitaciones, superficie, disponibilidad de parking,
6  * estado de conservación y propietario.
7  */
8 public class Vivienda { 2 usages
9     /** Precio de la vivienda en euros */
10    private double precio; 4 usages
11
12    /** Número de habitaciones de la vivienda */
13    private int numHabitaciones; 4 usages
14
15    /** Superficie de la vivienda en metros cuadrados */
16    private double superficie; 4 usages
17
18    /** Indica si la vivienda tiene parking (true = sí, false = no) */
19    private boolean parking; 4 usages
20
21    /** Estado actual de la vivienda (ejemplo: "Nuevo", "Usado", "Reformado") */
22    private String estado; 4 usages
23
24    /** Nombre del propietario de la vivienda */
25    private String propietario; 4 usages
26
27    /** Descuento aplicado al precio de la vivienda */
28    private int descuento; 1 usage
29
30    /**
31     * Constructor que inicializa la vivienda con un precio y aplica un descuento.
32     *
33     * @param precio Precio inicial de la vivienda.
34     */
35    public Vivienda(double precio) { 1 usage
36        this.setPrecio(precio, descuento: 0); // Se establece el descuento en 0 por defecto
37    }
38
39    /**
40     * Constructor que inicializa una vivienda con todos sus atributos.
41     *
42     * @param precio Precio de la vivienda.
```


Ejercicio 8.

Género la documentación javadoc:



The image shows two screenshots from an IDE. The top screenshot displays the 'Tools' menu with the 'Generate JavaDoc...' option highlighted. The bottom screenshot shows the Javadoc documentation for the 'Hogar' package and the 'Vivienda' class.

Package Hogar

package Hogar

Class	Description
Hogar	
Vivienda	La clase Vivienda representa una propiedad con diversas características, incluyendo precio, número de habitaciones, superficie, disponibilidad de parking, estado de conservación y propietario.

Package Hogar > Vivienda

Constructor

Constructor	Description
Vivienda(double precio)	Constructor que inicializa la vivienda con un precio y aplica un descuento.
Vivienda(double precio, int numHabitaciones, double superficie, boolean parking, String estado, String propietario)	Constructor que inicializa una vivienda con todos sus atributos.

Method Summary

Modifier and Type	Method	Description
String	getEstado()	Obtiene el estado actual de la vivienda.
int	getNumHabitaciones()	Obtiene el número de habitaciones de la vivienda.
double	getPrecio()	Obtiene el precio de la vivienda.
String	getPropietario()	Obtiene el propietario de la vivienda.
double	getSuperficie()	Obtiene la superficie de la vivienda.
boolean	isParking()	Indica si la vivienda tiene parking.
void	setEstado(String estado)	Establece un nuevo estado para la vivienda.
void	setNumHabitaciones(int numHabitaciones)	Establece un nuevo número de habitaciones para la vivienda.
void	setParking(boolean parking)	Define si la vivienda tiene parking o no.
void	setPrecio(double precio, int descuento)	Establece un nuevo precio para la vivienda, aplicando un descuento si corresponde.
void	setPropietario(String propietario)	Establece el propietario de la vivienda.
void	setSuperficie(double superficie)	Establece la superficie de la vivienda.
String	toString()	Representación en cadena de la vivienda, mostrando todos sus atributos.

Methods inherited from class java.lang.Object

clone(), equals(), finalize(), getClass(), hashCode(), notify(), notifyAll(), wait(), wait(), wait()